



Rapport de Stage

Ahaddou Heymad
Dev FullStack
du 05/01/2026 au 12/02/2026
2SLAM

Tables des matières

Introduction.....	4
Présentation de l'organisation d'accueil.....	5
Activités principales.....	5
Outils collaboratifs et méthodes de travail.....	5
Contexte du stage.....	5
Contexte du stage : Le projet "L'islam dans l'histoire".....	5
Stack technique : Architecture Full Stack Moderne et Performante.....	6
Travail en autonomie.....	7
Moyens mis à disposition.....	7
Équipement informatique.....	7
Stack technique et outils de collaboration.....	7
Missions réalisées.....	8
Présentation des missions.....	8
Projets et applications développés : "L'islam dans l'histoire".....	8
Fonctionnalités développées.....	8
A. Sécurisation de l'infrastructure et des données.....	8
1. Gestion des variables d'environnement (AdminJS).....	8
B. Étude et prototypage d'une carte interactive (Leaflet).....	10
1. Veille technologique et choix de la solution.....	10
2. Conception de la maquette fonctionnelle.....	10
C. Suivi utilisateur et expérience personnalisée (UX).....	11
1. Évolution du modèle de données (MongoDB).....	11
2. Logique métier : Gestion de l'état "Lu".....	12
Livrables produits.....	12
A. Code Source et Versioning (Bitbucket).....	12
B. Documentation Technique et Configuration.....	12
C. Évolution de la Base de Données (MongoDB).....	12
D. Suivi de Gestion de Projet (Trello).....	13
Difficultés rencontrées et solutions apportées.....	13
Défis techniques principaux.....	13
Découverte de nouvelles technologies.....	13
Stratégies de résolution mises en œuvre.....	14
Bilan des solutions apportées.....	14
Outils, ressources et compétences mobilisées.....	15
Environnements logiciels et matériels.....	15
Méthodologies et pratiques professionnelles.....	15
Compétences acquises.....	15
Bilan de l'expérience de stage.....	15
Compétences techniques acquises.....	15
Compétences méthodologiques.....	16
Compétences transversales.....	16
Conclusion.....	16
Apports professionnels.....	16

Apports personnels.....	17
Développement de l'autonomie d'apprentissage.....	17

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réussite de mon stage au sein de l'entreprise Dakilab, du 05 janvier au 12 février 2026.

Mes premiers remerciements s'adressent à **Monsieur Abdellatif El Maknati**, mon maître de stage, qui m'a accueilli dans l'entreprise et m'a accompagné tout au long de cette période. Sa disponibilité, ses conseils avisés et son encadrement bienveillant ont été déterminants pour mon apprentissage et ma compréhension du métier.

Enfin, je remercie l'ensemble du personnel de Dakilab pour leur accueil chaleureux et pour avoir fait de ce stage une expérience humaine et professionnelle enrichissante.

Cette période de stage m'a permis de développer mes compétences techniques, de découvrir le monde de l'entreprise et de confirmer mon orientation professionnelle dans le domaine informatique.

Introduction

Dans le cadre de ma première année de BTS Services Informatiques aux Organisations (SIO), option **Solutions Logicielles et Applications Métiers (SLAM)**, j'ai effectué un stage de six semaines, du 05 janvier au 12 février **2026**, au sein de **Dakilab**

Ce stage constitue une étape charnière de ma formation. Il m'a permis de sortir du cadre académique pour m'immerger dans un projet concret et ambitieux : la plateforme web "**L'islam dans l'histoire**". Ce projet, mêlant médiation culturelle et outils numériques, représentait un terrain d'apprentissage idéal pour approfondir mes compétences en développement **Full Stack**.

Mes attentes pour cette immersion étaient multiples. Au-delà de la découverte du fonctionnement d'une équipe de développement, je souhaitais :

- Appréhender le cycle de vie d'une application web, de la gestion de base de données (**MongoDB**) via l'outil AdminJS à l'interface utilisateur (**Angular**).
- Comprendre les enjeux de sécurité et de robustesse d'un backend moderne avec **NestJS**.
- Confronter mes connaissances théoriques aux contraintes réelles du métier (qualité du code, gestion des versions avec Git, respect de l'expérience utilisateur).

Ce rapport présente, dans un premier temps, l'organisation d'accueil et le contexte technologique du projet. Ensuite, je détaillerai la synthèse des missions qui m'ont été confiées — notamment la sécurisation des accès et la mise en place d'outils interactifs comme la cartographie dynamique. Enfin, je proposerai un bilan sur les compétences acquises et l'apport de cette expérience pour mon futur projet professionnel.

Présentation de l'organisation d'accueil

Activités principales

Dakilab est un studio de création numérique basé à **Nantes**. L'entreprise se définit comme un laboratoire d'innovation digitale, accompagnant ses clients (startups, PME ou grands groupes) dans la conception et le développement de solutions web et mobiles sur mesure.

Contrairement à une ESN classique, Dakilab met l'accent sur une approche "studio" où le design (UX/UI) et la qualité technique sont au cœur de chaque projet. L'agence intervient sur l'ensemble du cycle de vie des produits : du conseil stratégique au déploiement technique, en passant par le prototypage.

Outils collaboratifs et méthodes de travail

Pour assurer le suivi des projets et la fluidité des développements, l'entreprise s'appuie sur des outils de gestion modernes :

- **Trello** : utilisé pour la gestion de projet en mode Agile. Chaque ticket (fonctionnalité, bug, amélioration) est suivi de sa création jusqu'à sa validation finale, permettant une visibilité claire sur l'avancement des tâches.
- **Bitbucket** : plateforme de gestion de versions basée sur Git. L'ensemble du code source du projet y est centralisé, permettant une gestion rigoureuse des branches de développement et des Pull Requests.
- **Communication interne** : l'équipe utilise des outils de messagerie instantanée pour les échanges quotidiens, complétés par des points techniques réguliers avec le maître de stage.

Contexte du stage

Contexte du stage : Le projet "L'islam dans l'histoire"

Mon stage a été intégralement centré sur la contribution et le développement de la plateforme innovante "**L'islam dans l'histoire**". Ce projet numérique de grande envergure a pour vocation essentielle d'être une ressource interactive et pédagogique de référence, destinée à un large public (étudiants, chercheurs, grand public), offrant un accès structuré et enrichi à des contenus historiques complexes. L'objectif principal est de démocratiser la compréhension de l'histoire de l'islam à travers une interface utilisateur moderne et intuitive, qui ne se contente pas de présenter des informations, mais incite également à l'exploration et à l'analyse critique.

Mon stage a été intégralement centré sur la contribution et le développement de la plateforme innovante "**L'islam dans l'histoire**", un projet numérique de grande envergure commandité par la Mosquée de Nantes à Dakilab. Ce projet a pour vocation essentielle d'être une ressource interactive et pédagogique de référence, destinée à un large public

(étudiants, chercheurs, grand public), offrant un accès structuré et enrichi à des contenus historiques complexes. L'objectif principal est de démocratiser la compréhension de l'histoire de l'islam à travers une interface utilisateur moderne et intuitive, qui ne se contente pas de présenter des informations, mais incite également à l'exploration et à l'analyse critique.

Mon rôle dans ce contexte a été crucial pour la mise en œuvre technique des fonctionnalités clés permettant d'atteindre cette ambition pédagogique et interactive. **Environnement et Méthodologie de travail : Riguer et Collaboration**

L'organisation du travail a reposé sur des outils et des méthodologies éprouvées, garantissant une gestion de projet efficace et une collaboration fluide au sein de l'équipe de développement :

- **Système de gestion de versions (VCS)** : L'ensemble du code source était géré via **Bitbucket**, en utilisant une stratégie de **Git flow** rigoureuse. Cette approche a permis d'assurer une traçabilité complète des modifications, une gestion sécurisée des branches de développement (feature, develop, main) et une intégration continue des contributions de chaque membre de l'équipe.
- **Gestion des tâches et agilité** : Le suivi de l'avancement et la répartition des tâches étaient orchestrés sur **Trello**, en appliquant la méthodologie **Kanban**. Cette visualisation claire du flux de travail (À faire, En cours, En revue, Terminé) a facilité la priorisation, l'identification rapide des goulots d'étranglement et a maintenu une forte réactivité face aux besoins fonctionnels du projet.

Stack technique : Architecture Full Stack Moderne et Performante

La plateforme a été conçue sur une architecture technique "Full Stack" choisie pour sa robustesse, sa modularité et sa capacité à supporter une croissance future :

- **Frontend (Interface Utilisateur)** : L'interface utilisateur interactive et dynamique a été développée avec **Angular**. Ce framework JavaScript moderne a permis de construire une Application Page Unique (SPA) performante, offrant une expérience utilisateur fluide et réactive, essentielle pour la navigation dans des contenus historiques volumineux et interconnectés.
- **Backend (Logique Applicative et API)** : Le cœur du système, gérant la logique métier, l'authentification et l'accès aux données, a été implémenté avec **NestJS**. Ce framework Node.js basé sur TypeScript est réputé pour sa structure modulaire et son utilisation des bonnes pratiques de l'architecture logicielle (comme les Design Patterns et l'injection de dépendances), permettant de développer des API RESTful robustes et maintenables.
- **Base de données (Stockage des Données)** : Le choix s'est porté sur **MongoDB**, une base de données NoSQL orientée documents. Cette flexibilité a été un atout majeur pour gérer la diversité et la nature non-relationnelle des données historiques (documents, médias, métadonnées, chronologies), facilitant l'évolution du schéma de données en fonction des besoins du contenu.

Travail en autonomie

L'organisation du travail chez **Dakilab** a été marquée par une forte dimension d'autonomie, le stage s'étant déroulé quasi intégralement en **distanciel**. Sous la supervision d'**Abdellatif El Maknati**, dirigeant de l'entreprise, j'ai dû mettre en place une méthodologie de travail rigoureuse pour assurer la progression du projet "L'islam dans l'histoire".

L'organisation du travail chez **Dakilab**, une entreprise à petite effectif, a été marquée par une forte dimension d'autonomie, le stage s'étant déroulé quasi intégralement en **distanciel** (ce qui est courant dans les petites structures). Sous la supervision d'**Abdellatif El Maknati**, dirigeant de l'entreprise, j'ai dû mettre en place une méthodologie de travail rigoureuse pour assurer la progression du projet "L'islam dans l'histoire".

Pour structurer cette collaboration à distance, nous avons instauré deux rituels quotidiens :

- **Le point du matin (Morning Call)** : Un échange pour définir les objectifs de la journée, prioriser les tâches sur Trello et lever les éventuels blocages techniques.
- **Le point du soir (Evening Check)** : Une revue des avancées effectuées, permettant de valider les fonctionnalités développées.

Mon flux de travail suivait un cycle précis : je récupérais mes tickets sur **Trello**, développais les fonctionnalités localement, puis soumettais mon code via des **Pull Requests (PR)** sur **Bitbucket**. Abdellatif El Maknati effectuait alors une **revue de code** (Code Review). En cas de besoin, il ajoutait des commentaires directement sur Bitbucket pour m'orienter vers des corrections ou des optimisations. Cette méthode m'a permis de développer une grande autonomie tout en garantissant la qualité technique du projet.

Moyens mis à disposition

Équipement informatique

Pour ce stage, j'ai utilisé mon équipement personnel sous Windows, sur lequel l'environnement de développement était déjà configuré. Cette configuration m'a permis d'être immédiatement opérationnel dès le début du stage. Mon outil principal de développement a été Visual Studio Code (VS Code), complété par l'utilisation de terminaux pour la gestion des commandes Git et des serveurs locaux.

Stack technique et outils de collaboration

Les technologies exploitées durant ce stage sont au cœur des standards actuels du développement web, garantissant la robustesse et la scalabilité de l'application :

- Frontend : Angular, pour la création d'une interface utilisateur dynamique et modulaire.
- Backend : NestJS, pour une architecture serveur structurée et performante.
- Base de données : MongoDB, permettant une gestion flexible des données (NoSQL).
- Administration : AdminJS, un framework d'administration permettant de gérer les données de l'application via une interface utilisateur simple et configurable.

- Gestion de versions : Bitbucket (Git), essentiel pour le suivi du code et la collaboration asynchrone via les Pull Requests.
- Gestion de projet : Trello, pour le suivi des tâches selon la méthode Kanban.

L'accès complet à ces outils professionnels m'a permis de travailler dans des conditions réelles de production, identiques à celles d'un développeur permanent en agence web.

Missions réalisées

Présentation des missions

Durant ces six semaines, mes missions ont été orientées vers le développement **Full Stack** de la plateforme "L'islam dans l'histoire". Mon travail s'est articulé autour de trois axes principaux :

1. **Le renforcement de la sécurité** des accès et des formulaires.
2. **L'amélioration de l'expérience utilisateur (UX/UI)** par la refonte d'interfaces et la correction de bugs ergonomiques.
3. **Le développement de fonctionnalités métier** complexes (cartographie et historique).

Projets et applications développés : "L'islam dans l'histoire"

Le projet est une application web moderne (SPA) permettant de consulter des ressources historiques.

- **Côté Backend (NestJS)** : Gestion des API REST, sécurisation des routes et interaction avec la base de données MongoDB.
- **Côté Frontend (Angular)** : Création de composants réutilisables, gestion de l'état de l'application et intégration de bibliothèques tierces.

Fonctionnalités développées

A. Sécurisation de l'infrastructure et des données

L'un des enjeux majeurs de mon stage a été de fiabiliser l'application face aux vulnérabilités courantes (OWASP) et de garantir l'étanchéité des données sensibles.

1. Gestion des variables d'environnement (AdminJS)

Afin de ne pas exposer d'identifiants sensibles sur le dépôt **Bitbucket**, j'ai procédé au découplage de la configuration et du code source. J'ai extrait les identifiants de l'interface d'administration pour les isoler dans un fichier `.env`, non versionné.

Pour consommer ces données de manière sécurisée dans l'architecture **NestJS**, j'ai injecté le **ConfigService** dans le module d'administration.

```
291 },
292 auth: {
293   authenticate: (email: string, password: string) => {
294     const adminPassword = configService.get<string>('ADMIN_PASSWORD');
295     const adminLogin = configService.get<string>('ADMIN_LOGIN');
296     if (
297       email === adminLogin &&
298       password === adminPassword
299     ) {
300       return { email };
301     }
302     return null;
303   },
304   cookieName: 'adminjs',
305   cookiePassword: 'secret',
306 },
307 sessionOptions: {
308   resave: true,
309   saveUninitialized: true,
310   secret: 'secret',
311 },
312 };
```

L'utilisation de **configService.get** permet de récupérer la valeur au runtime, assurant que le mot de passe n'apparaît jamais "en clair" dans les fichiers compilés ou le gestionnaire de version.

2. Protection contre le spam (Captcha)

Pour protéger les formulaires de l'application (contact et inscription) contre les soumissions automatisées, j'ai implémenté un système de vérification par **Captcha**. La sécurité repose sur une validation en deux étapes : le Frontend génère un jeton (token) que le Backend doit impérativement valider auprès d'un fournisseur tiers (Google ou Cloudflare) avant de persister les données en base.

J'ai conçu un service capable de s'adapter au fournisseur configuré dans les variables d'environnement, garantissant ainsi la portabilité de la solution.

src/contact/contact.service.ts

```

39  /**
40   * Vérifie le token captcha auprès du fournisseur (Google reCAPTCHA ou Cloudflare Turnstile)
41   */
42   private async verifyCaptcha(token: string): Promise<boolean> {
43     if (!token) return false;
44
45     const provider = (process.env.CAPTCHA_PROVIDER || 'google').toLowerCase();
46     const secret = process.env.CAPTCHA_SECRET_KEY;
47
48     if (!secret) {
49       throw new BadRequestException(
50         "CAPTCHA non configuré sur le serveur. Contactez l'administrateur.",
51       );
52     }
53
54     try {
55       const axios = (await import('axios')).default;
56
57       const url =
58         provider === 'cloudflare'
59           ? 'https://challenges.cloudflare.com/turnstile/v0/siteverify'
60           : 'https://www.google.com/recaptcha/api/siteverify';
61
62       const params = new URLSearchParams();
63       params.append('secret', secret);
64       params.append('response', token);
65
66       const resp = await axios.post(url, params.toString(), {
67         headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
68         timeout: 5000,
69       });
70
71       return !!resp?.data?.success;
72     } catch {
73       // Fail closed en cas d'erreur réseau
74       return false;
75     }
76   }
77 }
78

```

B. Étude et prototypage d'une carte interactive (Leaflet)

L'un des objectifs du stage était de définir une solution technique permettant d'ajouter une dimension spatiale aux récits historiques. Ma mission a consisté à effectuer une veille technologique et à concevoir un prototype interactif répondant aux besoins du projet.

1. Veille technologique et choix de la solution

Après avoir comparé différentes solutions (Google Maps, Mapbox, Leaflet), j'ai préconisé l'utilisation de **Leaflet**. Ce choix se justifie par :

- **La gratuité et l'aspect Open Source** : Pas de clés d'API payantes.
- **La légèreté** : Excellentes performances pour une application web mobile.
- **La personnalisation** : Capacité à utiliser des fonds de carte (tuiles) spécifiques.

Pour l'aspect visuel, j'ai sélectionné les tuiles **Esri World Shaded Relief**. Ce fond de carte "ombré" offre un rendu sobre et élégant, parfaitement adapté à une thématique historique.

2. Conception de la maquette fonctionnelle

Afin de valider la faisabilité technique auprès de mon maître de stage, j'ai développé un prototype autonome. L'enjeu principal était de restreindre l'interaction utilisateur à la zone géographique concernée par le podcast.

J'ai utilisé la propriété `maxBounds` pour "verrouiller" la vue sur une zone précise (entre la péninsule arabique et le bassin méditerranéen), évitant ainsi que l'utilisateur ne s'égaré sur des zones hors sujet.

```

25 <div id="map"></div>
26 <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
27 <script>
28   const map = L.map('map', {
29     center: [25, 47],
30     zoom: 5,
31     minZoom: 5,
32     maxZoom: 18,
33     // Limiter la zone VISIBLE (pan restreint) mais garder toutes les tuiles du monde
34     maxBounds: [[10, 25], [40, 70]],
35     maxBoundsViscosity: 1.0,
36     scrollWheelZoom: true,
37     doubleClickZoom: true,
38     boxZoom: true,
39     zoomControl: true,
40     dragging: true
41   });
42
43   L.tileLayer('https://server.arcgisonline.com/ArcGIS/rest/services/World_Shaded_Relief/MapServer/tile/{z}/{y}
44     maxZoom: 18,
45     attribution: '© Esri',
46     detectRetina: true,
47     keepBuffer: 4,
48     updateWhenZooming: true,
49     updateWhenIdle: false
50     // PAS de 'bounds' ici = tuiles du monde entier disponibles
51   }).addTo(map);
52

```

C. Suivi utilisateur et expérience personnalisée (UX)

L'engagement des utilisateurs sur une plateforme de contenus repose sur la capacité de celle-ci à proposer un suivi de progression. J'ai donc travaillé sur l'implémentation d'un système d'historique permettant de distinguer les contenus consultés ("Lu") de ceux restant à découvrir.

1. Évolution du modèle de données (MongoDB)

Pour persister l'historique de chaque utilisateur, j'ai dû modifier le schéma `User` côté Backend. L'enjeu était de stocker une liste d'identifiants de fiches "Personnalités" de manière efficace, sans alourdir les requêtes de connexion.

J'ai opté pour l'ajout d'un tableau d'identifiants (Strings ou ObjectIds) au sein de la collection

```

37     newsletter?: boolean;
38
39     @Prop({ type: [String], default: [] })
40     readPersonalities: string[];
41 }
42
43 export const UserSchema = SchemaFactory.createForClass(User);
44

```

2. Logique métier : Gestion de l'état "Lu"

Côté service (`UserService`), j'ai implémenté une méthode permettant d'alimenter cet historique. Pour garantir l'intégrité des données et éviter les doublons dans le tableau, j'ai utilisé l'opérateur MongoDB `$addToSet`.

```

69  /**
70   * Ajoute une personnalité à l'historique de l'utilisateur
71   */
72  async markAsRead(userId: string, personalityId: string): Promise<void> {
73      await this.userModel
74          .findByIdAndUpdate(userId, {
75              // $addToSet garantit que l'ID n'est ajouté que s'il n'existe pas déjà
76              $addToSet: { readPersonalities: personalityId },
77          })
78          .exec();
79  }

```

Cette approche permet une mise à jour atomique en base de données, minimisant les ressources serveur nécessaires.

Livrables produits

L'aboutissement de mes missions s'est traduit par la livraison de plusieurs éléments techniques et organisationnels, garantissant la pérennité des développements effectués pour **Dakilab**.

A. Code Source et Versioning (Bitbucket)

Le livrable principal est constitué par l'ensemble du code source produit pour les modules Frontend (Angular) et Backend (NestJS). Fidèle aux bonnes pratiques de l'entreprise, chaque fonctionnalité a été isolée dans une branche dédiée avant d'être soumise à une **Pull Request**.

- **Volume** : Une dizaine de Pull Requests validées et fusionnées (mergées) après revue de code.
- **Qualité** : Respect des normes de nommage TypeScript et modularité des composants.

B. Documentation Technique et Configuration

Afin de faciliter la reprise du code par d'autres développeurs et d'assurer le déploiement sur différents environnements (développement, production), j'ai produit :

- **Commentaires JSDoc** : Documentation intra-code pour les méthodes complexes (notamment la logique de validation Captcha).
- **Mise à jour du README** : Rédaction d'un guide pour la configuration des nouvelles variables d'environnement (`.env.example`), indispensables au fonctionnement d'AdminJS et du service de Captcha.

C. Évolution de la Base de Données (MongoDB)

J'ai livré une structure de données enrichie. Le schéma de la collection `User` a été mis à jour de manière persistante pour intégrer le suivi des ressources consultées. Ces modifications ont été répercutées et rendues administrables via l'interface **AdminJS**.

D. Suivi de Gestion de Projet (Trello)

Enfin, sur le plan organisationnel, le livrable consiste en un tableau de bord de projet à jour. L'intégralité des tickets qui m'ont été attribués a été documentée, traitée et déplacée dans la colonne « Terminé » (Done) après validation technique. Ce suivi permet à l'entreprise de conserver une traçabilité précise de l'évolution de la plateforme durant mon stage.

Difficultés rencontrées et solutions apportées

Défis techniques principaux

Le passage d'un environnement académique à un projet professionnel d'envergure chez **Dakilab** a soulevé plusieurs défis majeurs :

- **L'intégration et la recherche sur Leaflet** : Contrairement à une simple intégration de carte, le besoin était spécifique (restriction de zone, esthétique historique). La difficulté résidait dans la compréhension des projections géographiques et l'utilisation de la propriété `maxBounds` pour verrouiller la navigation, tout en garantissant la fluidité du rendu.
- **La sécurisation par Captcha** : Implémenter le Captcha a nécessité une compréhension parfaite du flux de données asynchrone. Le défi était de synchroniser la réponse du service tiers (Google/Cloudflare) avec notre propre backend NestJS pour ne valider le message en base qu'une fois le jeton certifié.
- **L'appréhension d'un code source existant** : Arriver sur une plateforme déjà structurée demande un effort d'analyse important. Il a fallu "rentre dans le code",

comprendre l'architecture en modules de NestJS et les composants Angular pour savoir exactement où intervenir sans casser l'existant.

Découverte de nouvelles technologies

Ce stage a marqué un tournant technique dans mon parcours avec l'apprentissage de plusieurs outils industriels :

- **Angular & TypeScript** : Bien que familier avec le JavaScript, la découverte du **TypeScript** a été un défi enrichissant. La gestion rigoureuse des types, des interfaces et des décorateurs (`@Component`, `@Injectable`) permet de produire un code beaucoup plus robuste et moins sujet aux erreurs.
- **AdminJS** : C'était ma première utilisation d'un "Automatic Admin Interface". J'ai dû apprendre à configurer les ressources et à adapter le schéma Mongoose pour que l'interface d'administration reflète parfaitement les besoins métiers (comme l'historique utilisateur).
- **Le lien NestJS / NuxtJS** : Mes connaissances préalables en **NuxtJS** (Framework Vue) m'ont aidé à comprendre la logique de **NestJS** (Framework Node). Bien qu'ils servent des buts différents, j'ai retrouvé des similitudes dans la gestion des routes, des services et de l'aspect modulaire, ce qui a facilité ma montée en compétence.

Stratégies de résolution mises en œuvre

Pour pallier le travail en **distanciel** et les blocages techniques, j'ai adopté une méthodologie structurée :

1. **Exploration de la documentation officielle** : Avant de solliciter mon maître de stage, j'ai pris le réflexe de consulter les documentations de Leaflet, NestJS et Angular. C'est la source la plus fiable pour comprendre les bonnes pratiques.
2. **Prototypage isolé** : Pour la carte Leaflet, j'ai d'abord créé un environnement de test minimal (HTML/JS simple) pour valider mes recherches avant de tenter l'intégration dans l'application Angular complexe.
3. **Utilisation de la Code Review** : Chaque Pull Request sur **Bitbucket** était l'occasion d'un échange avec **Abdellatif El Maknati**. Ses commentaires m'ont permis de corriger ma logique et d'apprendre des méthodes de développement plus optimisées.

Bilan des solutions apportées

L'ensemble de ces défis a trouvé une issue positive, apportant une réelle valeur ajoutée au projet :

- **Autonomie accrue** : Le travail en distanciel, loin d'être un frein, est devenu un moteur. Il m'a forcé à approfondir mes recherches par moi-même, renforçant ma capacité à résoudre des problèmes complexes de manière indépendante.

- **Code maintenable** : Les solutions apportées (variables d'environnement, validation Captcha robuste) ne sont pas de simples "patches", mais des fonctionnalités intégrées selon les standards NestJS, facilitant le travail des futurs développeurs.
- **Aide à la décision** : Mes recherches sur Leaflet ont permis de fixer le socle technique de la partie cartographique du projet, validant ainsi la faisabilité d'une fonctionnalité centrale de la plateforme.

Outils, ressources et compétences mobilisées

Environnements logiciels et matériels

Pour ce stage, j'ai travaillé sur mon ordinateur personnel sous Windows, ce qui m'a permis d'être immédiatement opérationnel dans un environnement familier. Mon outil de travail principal était l'éditeur VS Code, complété par Git pour le versionnage du code. L'ensemble du projet était centralisé sur Bitbucket, où je pouvais gérer mes différentes branches de développement. L'interface AdminJS a également été une ressource clé pour suivre l'évolution de la base de données MongoDB en temps réel.

Méthodologies et pratiques professionnelles

Le travail en distanciel a nécessité une organisation rigoureuse basée sur des échanges quotidiens via Slack et des visioconférences. Nous avons fonctionné avec un système de "Morning Calls" pour définir les tâches et de points le soir pour valider les avancées. J'ai utilisé Trello pour gérer mon tableau de bord et suivre l'état de mes tickets (en cours, à tester, terminé). Enfin, chaque fonctionnalité a suivi un processus de revue de code systématique sur Bitbucket avant d'être intégrée au projet final.

Compétences acquises

Ce stage m'a permis de monter en compétence sur des technologies modernes comme Angular et NestJS, tout en renforçant ma maîtrise du TypeScript. J'ai appris à structurer une application Full Stack et à modifier des schémas de base de données NoSQL. Au-delà de l'aspect technique, j'ai surtout gagné en autonomie : le fait d'être à distance m'a appris à analyser les problèmes par moi-même, à consulter efficacement les documentations et à proposer des solutions avant de solliciter mon maître de stage.

Bilan de l'expérience de stage

Compétences techniques acquises

- **Maîtrise d'une stack moderne** : Développement Full Stack complet avec **Angular** pour le front et **NestJS** pour le back.
- **Utilisation du TypeScript** : Passage du JavaScript classique à un langage typé, plus rigoureux et adapté aux gros projets.

- **Gestion de base de données NoSQL** : Manipulation et mise à jour de schémas avec **MongoDB** et Mongoose.
- **Sécurisation applicative** : Mise en place de systèmes de protection contre le spam via des solutions de **Captcha**.
- **Administration simplifiée** : Configuration d'un back-office automatisé avec **AdminJS** pour la gestion des contenus.

Compétences méthodologiques

- **Workflow professionnel (Git)** : Utilisation de **Bitbucket** pour gérer les versions, avec un cycle complet (branches, Pull Requests et Code Reviews).
- **Gestion de projet Agile** : Organisation du travail via **Trello** en suivant la méthode Kanban (découpage en tickets).
- **Rituels de travail à distance** : Mise en place d'une routine efficace avec des "Daily Meetings" le matin et le soir pour le suivi de production.
- **Documentation technique** : Rédaction de fichiers README et commentaires de code pour assurer la maintenance future du projet.

Compétences transversales

- **Forte autonomie** : Capacité à avancer seul en distanciel tout en sachant solliciter de l'aide au bon moment sur Slack.
- **Esprit d'analyse et de recherche** : Réalisation de vieilles techniques pour proposer des solutions (comme pour le prototype Leaflet).
- **Adaptation au code existant** : Capacité à comprendre et à s'intégrer dans un projet complexe déjà commencé par d'autres développeurs.
- **Communication professionnelle** : Échanges clairs et réguliers avec mon maître de stage pour rendre compte des avancées et des blocages.

Conclusion

Apports professionnels

Ce stage a marqué un véritable bond en avant technique dans mon parcours. En travaillant sur une stack aussi moderne que le combo Angular, NestJS et MongoDB, j'ai quitté le confort des projets scolaires pour me confronter à une architecture logicielle de niveau professionnel. L'utilisation rigoureuse du TypeScript m'a permis de comprendre l'importance du typage pour la robustesse d'une application, tandis que l'intégration d'outils comme AdminJS ou le système de Captcha m'a sensibilisé aux problématiques concrètes de sécurité et de gestion de données.

Au-delà de la technique, cette expérience m'a apporté une vision métier très claire. J'ai compris que le rôle d'un développeur en agence ne se limite pas à écrire du code, mais consiste à répondre à un contexte d'usage précis. Pour le projet "L'islam dans l'histoire", chaque fonctionnalité, qu'il s'agisse de la carte interactive Leaflet ou du système

d'historique, devait être pensée pour servir l'expérience utilisateur et la pédagogie. Travailler chez Dakilab m'a montré comment l'innovation technique se met au service d'un contenu éditorial.

Apports personnels

Sur le plan personnel, ce stage a été un moteur pour le **développement de mon autonomie d'apprentissage**. Le format en distanciel, bien que complexe au début, a été une opportunité pour apprendre à chercher l'information par moi-même. J'ai pris l'habitude de plonger dans les documentations officielles et de tester mes idées via des prototypes avant de solliciter mon maître de stage. Cette capacité à s'auto-former est, je pense, l'une des compétences les plus précieuses pour un futur développeur.

En conclusion, ces six semaines m'ont permis de gagner en maturité et en confiance. Je repars de chez Dakilab avec une méthodologie de travail solide, basée sur la rigueur du **Git Flow** et de la **Code Review**, et avec la certitude que le développement Full Stack est la voie dans laquelle je souhaite poursuivre ma carrière professionnelle.

Développement de l'autonomie d'apprentissage

L'un des apports les plus significatifs de ce stage a été le développement de mon **autonomie d'apprentissage**. Le format du travail en distanciel a agi comme un accélérateur : loin d'être un obstacle, cela m'a poussé à chercher des solutions par moi-même avant de solliciter l'aide de mon maître de stage. J'ai pris l'habitude de plonger dans les documentations officielles (Angular, NestJS, Leaflet) et de tester des idées via des prototypes isolés pour comprendre le fonctionnement de chaque outil.

Cette capacité à m'auto-former et à diagnostiquer mes propres erreurs est une compétence qui me sera indispensable pour la suite de ma carrière. Le métier de développeur demandant une veille technologique constante, ce stage m'a prouvé que j'étais capable de monter rapidement en compétence sur des technologies que je ne maîtrisais pas au départ. Cela m'a donné une plus grande confiance dans ma faculté à aborder des problèmes complexes de manière calme et méthodique.